

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	20% of CIA	Total Marks:	100 (2 Questions × 50 Marks)
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	K Tejaswi Reddy	Reg. No.:	192371036
Section / Batch:	CS-BIOSCI	Date:	29-08-2026

Instructions to Students

- Answer both questions. Each question carries 50 Marks. Total: 100 Marks.
- Carefully analyze the given questions.
- Explain in detail with sample code if needed.
- Clearly identifies all key issues.
- Thorough analysis with validated results and performance evaluation

Question Type	Description	Questions	Total Marks
Industry Problem Solving Task	Focus on Solving optimization problems using dynamic programming and greedy strategies	Q1 – Q2	100

SECTION A – Industry Problem Solving Task

Home - Classroom

SIMATS Online Lab

SIMATS Vana Study Portal

Ask Gemini

Not secure simatslab.saveetha.com/practice_app/classactivity/attempt/assignment_id=1853

Search

School



SIMATS Online Programming Lab
DAA PROGRAMMING

KOMMURU TEJASWI REDDY
19237020402

QUESTIONS

Q1
Greedy Technique
30 marks

Q2
Greedy Technique
30 marks

2/2 answered

01 Greedy Technique

30 marks

TSP - Logistics Optimization
A delivery company must visit 15 cities exactly once and return to the depot while minimizing travel cost. Analyze how the Travelling Salesman Problem can be solved using Dynamic Programming (Held-Karp algorithm). Design a solution by explaining state representation and recurrence relation. Discuss the feasibility of this approach in real-world logistics. Suggest appropriate tools for implementation. Analyze scalability issues as the number of cities increases and evaluate performance limitations.

Your Code

```
1 import itertools
2
3 def held_karp(dist):
4     n = len(dist)
5     dp = {}
6     for k in range(1, n):
7         dp[k] = {}
8         for subset_size in range(0, n):
9             for subset in itertools.combinations(range(0, n), subset_size):
10                 bits = 0
11                 for bit in subset:
12                     bits |= 1 << bit
13                 for k2 in subset:
14                     prev_bits = bits <~ (1 << k2)
15                     res = {}
16                     for w in subset:
17                         if w == k2:
18                             continue
19                     res[bits | (1 << k2)] = min(res.get(bits | (1 << k2), float('inf')), dp[prev_bits] + dist[k2][k])
20     return res[bits | (1 << n-1)]
```

Input
stdin input

Output
Number of cities (including depot): 15
Distance Matrix:
[[0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120, 130, 140],
[10, 0, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120, 130, 140],
[20, 20, 0, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120, 130, 140],
[30, 30, 30, 0, 40, 50, 60, 70, 80, 90, 100, 110, 120, 130, 140],
[40, 40, 40, 40, 0, 50, 60, 70, 80, 90, 100, 110, 120, 130, 140],
[50, 50, 50, 50, 50, 0, 60, 70, 80, 90, 100, 110, 120, 130, 140],
[60, 60, 60, 60, 60, 60, 0, 70, 80, 90, 100, 110, 120, 130, 140],
[70, 70, 70, 70, 70, 70, 70, 0, 80, 90, 100, 110, 120, 130, 140],
[80, 80, 80, 80, 80, 80, 80, 80, 0, 90, 100, 110, 120, 130, 140],
[90, 90, 90, 90, 90, 90, 90, 90, 90, 0, 100, 110, 120, 130, 140],
[100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 0, 110, 120, 130, 140],
[110, 110, 110, 110, 110, 110, 110, 110, 110, 110, 110, 0, 120, 130, 140],
[120, 120, 120, 120, 120, 120, 120, 120, 120, 120, 120, 120, 0, 130, 140],
[130, 130, 130, 130, 130, 130, 130, 130, 130, 130, 130, 130, 130, 0, 140],
[140, 140, 140, 140, 140, 140, 140, 140, 140, 140, 140, 140, 140, 140, 0]]

Activate Windows
Go to Settings to activate Windows.

Home - Classroom

SIMATS Online Lab

SIMATS Vana Study Portal

Ask Gemini

Not secure simatslab.saveetha.com/practice_app/classactivity/attempt/assignment_id=1853

Search

School



SIMATS Online Programming Lab
DAA PROGRAMMING

KOMMURU TEJASWI REDDY
19237020402

```
1 dp[bits, k] = min(res)
2 bits = (1 << n) - 1
3 res = {}
4 for k in range(1, n):
5     for (bits, k) in dp:
6         res.append((dp[bits, k][0] + dist[k][0], k))
7
8 opt_cost, parent = min(res)
9 return opt_cost
10
11 def generate_random_distance_matrix(n, seed=42):
12     """Generates a symmetric random distance matrix for n cities."""
13     import random
14     random.seed(seed)
15     dist = [[0] * n for _ in range(n)]
16     for i in range(n):
17         for j in range(i + 1, n):
18             d = random.randint(10, 100)
19             dist[i][j] = d
20             dist[j][i] = d
21     return dist
22
23 def main():
24     n = 15
25     dist_matrix = generate_random_distance_matrix(n)
26
27     print(f"Number of cities (including depot): {n}")
28     print("Distance Matrix:")
29     for row in dist_matrix:
30         print(row)
31
32     print("Running Held-Karp Algorithm...")
33     optimal_cost = held_karp(dist_matrix)
```

Activate Windows
Go to Settings to activate Windows.

Home - Classroom
SIMATS Online Lab
SIMATS Vana Study Portal
Ask Gemini
School

Not secure
simatslab.saveetha.com/practice_app/classactivity/attempt/assignment_id=1853

SIMATS ENGINEERING
SIMATS Online Programming Lab
DAA PROGRAMMING
KOMMURU TEJASWI REDDY

```

1 import random
2 def generate_random_distance_matrixs, seed=42):
3     """Generates a symmetric random distance matrix for n cities."""
4     import random
5     random.seed(seed)
6     dist = [[0] * n for _ in range(n)]
7     for i in range(n):
8         for j in range(i + 1, n):
9             d = random.randint(10, 100)
10            dist[i][j] = d
11            dist[j][i] = d
12        return dist
13
14 def main():
15     n = 10
16     dist_matrix = generate_random_distance_matrixs(n)
17
18     print(f"Number of cities (including depot): {n}")
19     print("Distance Matrix:")
20     for row in dist_matrix:
21         print(row)
22
23     print("Running Held-Karp Algorithm...")
24     optimal_cost = held_karp(dist_matrix)
25
26     print(f"Minimum Travel Cost for the optimal TSP tour: {optimal_cost}")
27
28 if __name__ == "__main__":
29     main()

```

Activate Windows
Go to Settings to activate Windows.

Home - Classroom
SIMATS Online Lab
SIMATS Vana Study Portal
Ask Gemini
School

Not secure
simatslab.saveetha.com/practice_app/classactivity/attempt/assignment_id=1853

SIMATS ENGINEERING
SIMATS Online Programming Lab
DAA PROGRAMMING
KOMMURU TEJASWI REDDY

QUESTIONS
Q1 Greedy Technique 10 marks
Q2 Greedy Technique 30 marks
3/2 answered

Q2 Greedy Technique 30 marks
Dijkstra Navigation System
A navigation system computes shortest paths between locations in real-time. Analyze how Dijkstra's algorithm can be applied to solve this problem efficiently. Design a solution by explaining step-by-step execution and the use of priority queues. Suggest appropriate data structures and tools for implementation. Analyze time complexity using different approaches. Interpret the performance in real-world navigation systems.

Your Code

```

1 import heapq
2
3 def dijkstra(graph, source, destination=None):
4     distances = {node: float('inf') for node in graph}
5     distances[source] = 0
6     previous = {node: None for node in graph}
7     visited = set()
8     pq = [(0, source)]
9
10    while pq:
11        current_dist, current_node = heapq.heappop(pq)
12        if current_node in visited:
13            continue
14        visited.add(current_node)
15        if destination and current_node == destination:
16            break
17
18        for neighbor, weight in graph[current_node].items():
19            distance = current_dist + weight
20            if distance < distances[neighbor]:
21                distances[neighbor] = distance
22                previous[neighbor] = current_node
23                heapq.heappush(pq, (distance, neighbor))

```

Input
stdin Input

Output
Shortest Distance from A to D is 9
Path: A -> B -> C -> D -> D

Activate Windows
Go to Settings to activate Windows.

```

14     for neighbor, weight in graph[current_node].items():
15         distance = current_dist + weight
16         if distance < distances[neighbor]:
17             distances[neighbor] = distance
18             previous[neighbor] = current_node
19             heapq.heappush(heap, (distance, neighbor))
20
21     return distances, previous
22
23 def reconstruct_path(previous, source, destination):
24     path = []
25     current = destination
26     while current is not None:
27         path.append(current)
28         current = previous[current]
29     path.reverse()
30     if path[0] == source:
31         return path
32     return None
33
34 graph = {
35     'A': {'B': 4, 'C': 1},
36     'B': {'A': 4, 'C': 2, 'D': 5},
37     'C': {'A': 1, 'B': 2, 'D': 8},
38     'D': {'B': 5, 'C': 8}
39 }
40
41 source = 'A'
42 destination = 'D'
43
44 distances, previous = dijkstra(graph, source, destination)
45 path = reconstruct_path(previous, source, destination)
46
47 print(f"Shortest distance from {source} to {destination}: {distances[destination]}")
48 print(f"Path: {' -> '.join(path)}")

```

Code of Conduct

I Tejaswi Reddy K (192371036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis	Good analysis with	Basic analysis	Poor or no analysis

		with validated results and performance evaluation	reasonable validation	with limited validation	
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Mark Evaluation:

Case Study	Understanding of Industry Problem (20)	Application of Engineering Concepts (25)	Solution Design (25)	Use of Tools (15)	Analysis (10)	Professionalism (5)	Total Marks (Each - 50)
Q1							
Q2							
	Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Name: _____ Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____